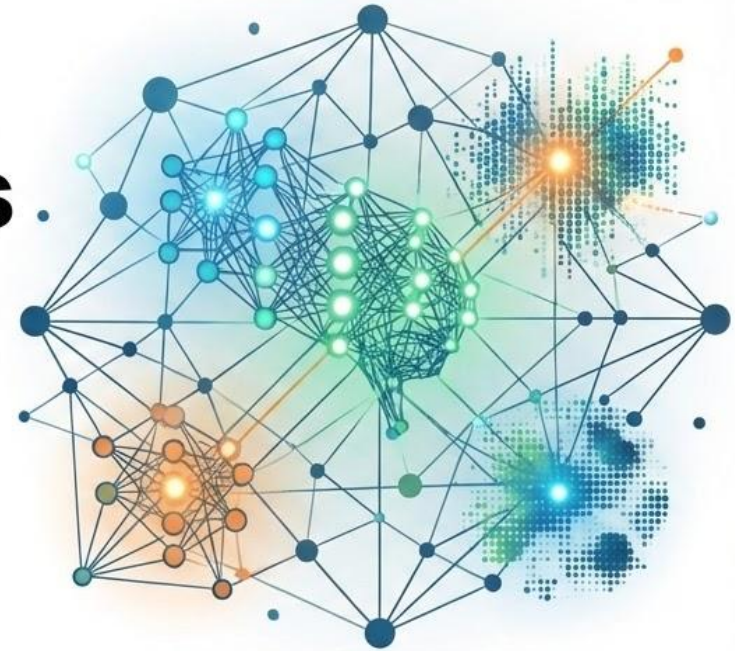


Diplomatura en python para la ciencia de datos y machine learning

Módulo 1 – Desarrollo
Avanzado en Python

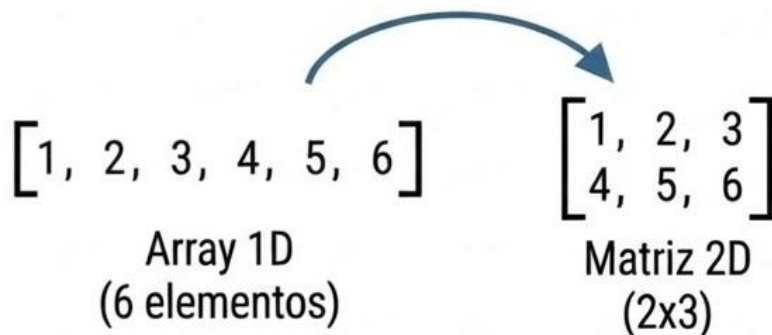


6.2- Computación Científica con NumPy

Parte 2: Transformación, combinación y agregación de datos; operaciones algebraicas y normalización; carga y guardado con NumPy.

Redimensionamiento de Arrays (Reshape) en NumPy

Concepto de Reshape



Cambia la forma sin cambiar los datos. El número total de elementos debe ser constante.

Sintaxis y Ejemplo

```
A = np.arange(1, 7) # [1, 2, 3, 4, 5, 6]
```

```
B = A.reshape((2, 3)) # [[1, 2, 3], [4, 5, 6]]
```

```
C = A.reshape((3, 2)) # [[1, 2],  
                        [3, 4], [5, 6]]
```

- ✓ Usar `-1` para calcular automáticamente
- ✓ una dimensión (ej. `A.reshape((-1, 2))` crea `3x2`).

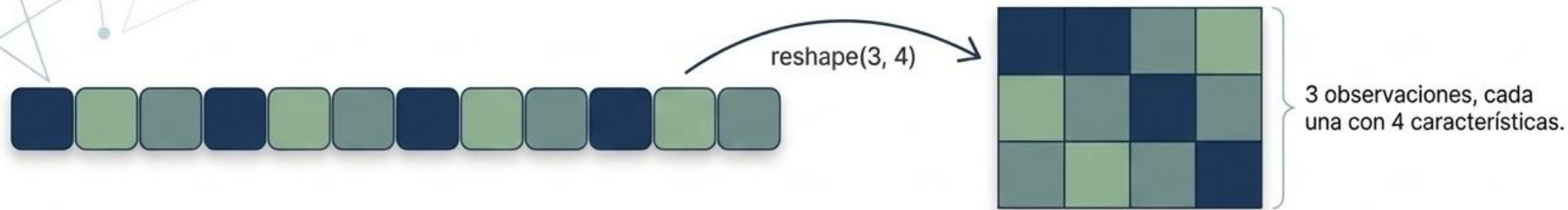


¡Importante! `reshape` retorna una nueva vista siempre que sea posible, sin copiar los datos.

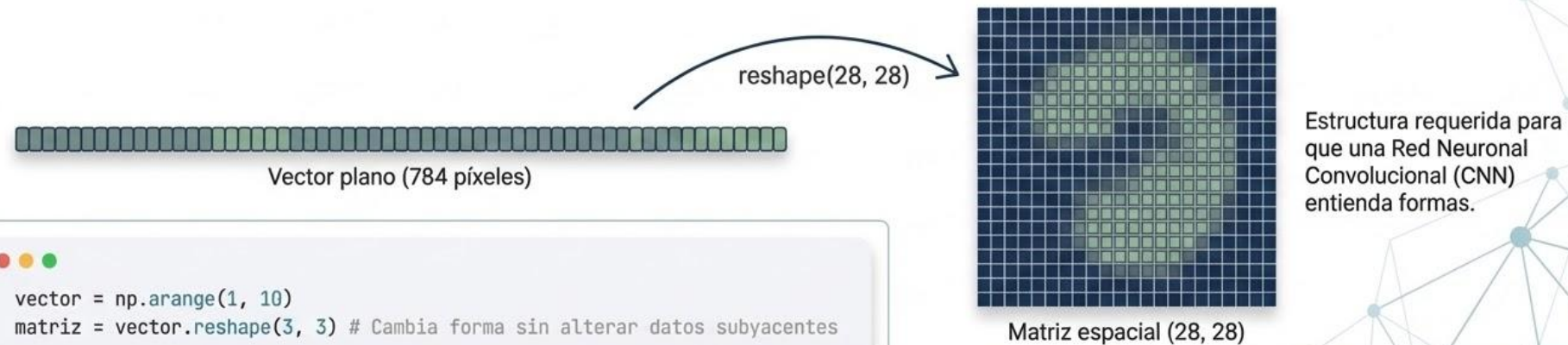
Moldeando la Realidad: El significado semántico de Reshape

reshape es el puente entre los datos crudos y la estructura exacta que un modelo predictivo espera recibir. Es una decisión de diseño, no solo estética.

1. Modelado ML Estándar



2. Visión Artificial



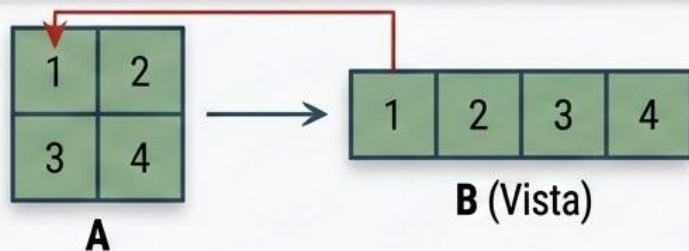
```
vector = np.arange(1, 10)
matriz = vector.reshape(3, 3) # Cambia forma sin alterar datos subyacentes
```

ravel vs flatten: Aplanamiento de Arrays en NumPy

ravel (Retorna una Vista)

Devuelve una **vista** (referencia) del array original siempre que sea posible. Es más eficiente en memoria. Modificar la vista afecta al original.

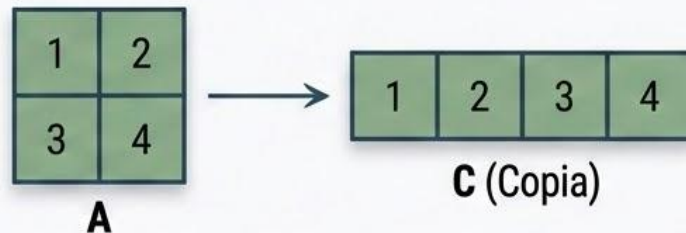
```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = A.ravel()
B[0] = 99
print(A) # Resultado: [[99, 2], [3, 4]] (A se modifica)
```



flatten (Retorna una Copia)

Siempre devuelve una **copia profunda** del array original. Usa más memoria. Modificar la copia **NO** afecta al original.

```
A = np.array([[1, 2], [3, 4]])
C = A.flatten()
C[0] = 99
print(A) # Resultado: [[1, 2], [3, 4]] (A no se modifica)
```

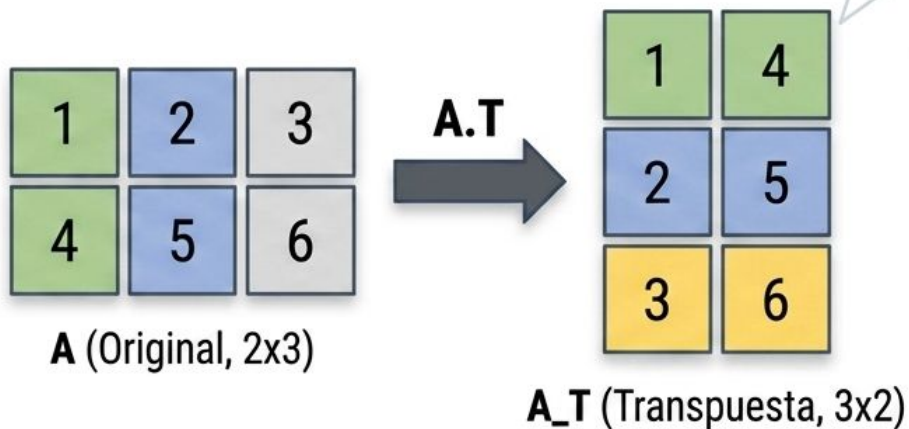


Ambos métodos aceptan el argumento opcional 'order' ('C', 'F', 'A', 'K') para especificar el orden de aplanamiento.

Transposición: Intercambiando Ejes con `.T` y `transpose`

La transposición invierte el orden de los ejes de un array. En una matriz 2D, las filas se convierten en columnas y viceversa. Es una operación fundamental en álgebra lineal y manipulación de datos.

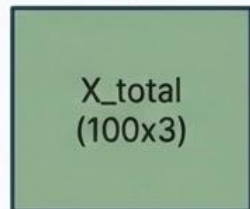
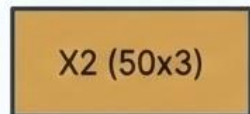
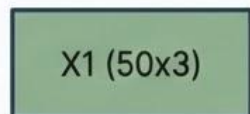
```
import numpy as np
A = np.array([[1, 2, 3],
              [4, 5, 6]]) # Forma (2, 3)
A_T = A.T # 0 np.transpose(A)
print(A_T)
# Resultado:
# [[1, 4],
#  [2, 5],
#  [3, 6]] # Forma (3, 2)
```



Nota: Al igual que ``ravel``, la transposición generalmente devuelve una **vista** (referencia) del array original, no una copia, lo que la hace muy **eficiente en memoria**.

Concatenación: Expandiendo el Dataset en el mundo real

Path 1: Apilamiento Vertical (axis=0)

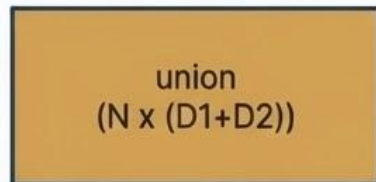
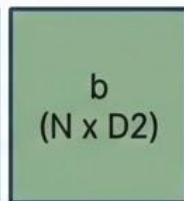
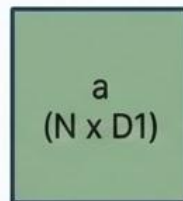


MÁS MUESTRAS

Añadir nuevas filas
(observaciones/personas)
al dataset existente.

```
X_total = np.vstack([X1, X2])
```

Path 2: Apilamiento Horizontal (axis=1)



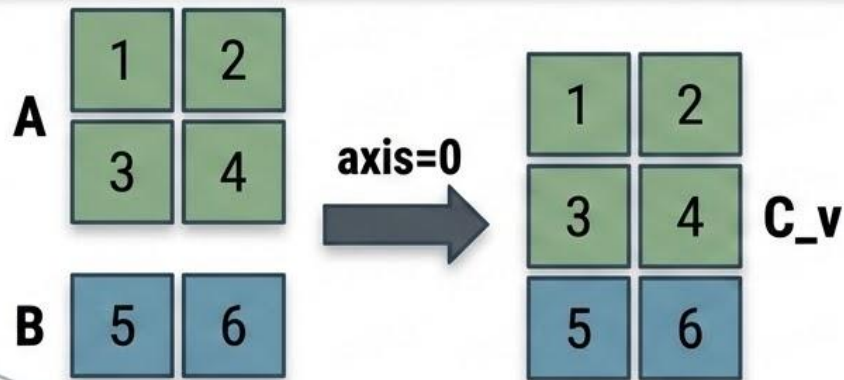
MÁS FEATURES

Enriquecer el dataset
añadiendo nuevas
columnas
(variables/métricas) a
las muestras existentes.

```
union = np.hstack((a, b))
```

Concatenación General: `np.concatenate`

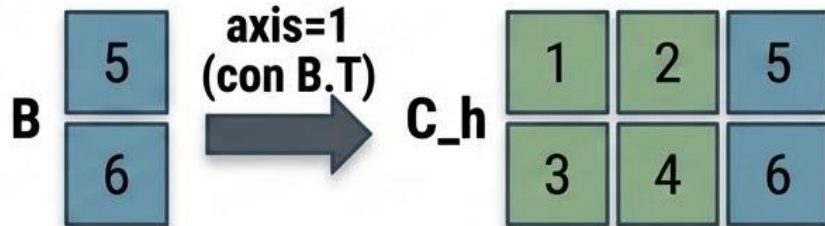
`np.concatenate` une una secuencia de arrays a lo largo de un eje existente. Es la función más general, donde `vstack` (**axis=0**) y `hstack` (**axis=1**) son casos especiales.



```
import numpy as np
A = np.array([[1, 2], [3, 4]]) # Forma (2, 2)
B = np.array([[5, 6]])         # Forma (1, 2)

# Eje 0 (Filas)
C_v = np.concatenate((A, B), axis=0)
# Resultado: [[1, 2], [3, 4], [5, 6]] (3, 2)

# Eje 1 (Columnas) - requiere transponer B
C_h = np.concatenate((A, B.T), axis=1)
# Resultado: [[1, 2, 5], [3, 4, 6]] (2, 3)
```

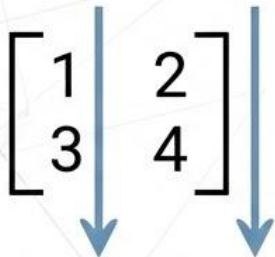


Nota: Todos los arrays de entrada deben tener la misma forma, excepto en la dimensión correspondiente al eje de concatenación.

Operaciones por Eje (Axis) en NumPy

Comprender cómo se aplican funciones a lo largo de las dimensiones de un array.

Eje 0: Columnas (Features)

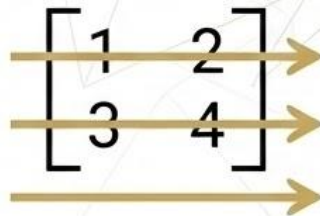


Operaciones **verticales**, a lo largo de las filas, para cada columna.

```
np.mean(X, axis=0)
```

$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \rightarrow \text{Resultado: } [2. \ 3.]$

Eje 1: Filas (Samples)



Operaciones **horizontales**, a lo largo de las columnas, para cada fila.

```
np.mean(X, axis=1)
```

$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \rightarrow \text{Resultado: } [1.5 \ 3.5]$



En **ciencia de datos y modelado**, las operaciones por columna (**axis=0**) son las más comunes para calcular estadísticas de características (features).

Agregaciones: Resumiendo Datos

Las funciones de agregación calculan estadísticas resumidas sobre arrays, ya sea globalmente o a lo largo de un eje específico.

- **Suma (`np.sum`)**
Calcula la suma total de los elementos.
- **Promedio (`np.mean`)**
Calcula el promedio aritmético.
- **Desviación Estándar (`np.std`)**
Mide la dispersión de los datos.
- **Mínimo (`np.min`)**
Encuentra el valor más pequeño.
- **Máximo (`np.max`)**
Encuentra el valor más grande.

```
np.sum(arr)
```

```
np.mean(arr, axis=0)
```

```
np.std(arr)
```

```
np.min(arr)
```

```
np.max(arr, axis=1)
```

Global: `np.sum(arr)` → 21



1	2	3
4	5	6

axis=1 (Filas)
`np.sum(arr, axis=1)`
→ [6, 15]



axis=0 (Columnas)
`np.sum(arr, axis=0)`
→ [5, 7, 9]

Funciones acumulativas: siguiendo la evolución de los datos

No resumen todo en un único valor → muestran el acumulado paso a paso

Funciones principales en NumPy:

np.cumsum() → suma acumulada

np.cumprod() → producto acumulado

Ejemplo:

```
import numpy as np

arr = np.array([1, 2, 3, 4])

np.cumsum(arr) # [1, 3, 6, 10]
np.cumprod(arr) # [1, 2, 6, 24]
```

¿Cuándo usarlo?

- ✓ Analizar crecimiento progresivo
- ✓ Ver evolución en el tiempo
- ✓ Preparar datos para análisis más complejos

Producto Punto vs. Producto Matricial: Entendiendo la Diferencia

Producto Punto (np.dot)

Operación algebraica que toma dos secuencias de números de igual longitud y devuelve un único número (escalar).

- Resulta en un **escalar**.

$$\begin{array}{l} A = \begin{bmatrix} a1 & a2 \end{bmatrix} \\ B = \begin{bmatrix} b1 & b2 \end{bmatrix} \end{array} \Rightarrow A \cdot B = a1*b1 + a2*b2$$

```
import numpy as np
a = np.array([1, 2])
b = np.array([3, 4])
np.dot(a, b) # Resultado: 11 (1*3 + 2*4)
```

- **Casos de uso:** Proyecciones, similitud entre vectores.

Producto Matricial (np.matmul / @)

Operación entre dos matrices que produce otra matriz. Las dimensiones internas deben coincidir.

- Resulta en una **matriz**.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \rightarrow \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
A @ B # Resultado: Matriz 2x2
```

- **Casos de uso:** Transformaciones lineales, redes neuronales.

Resumen Clave:

- **np.dot:** Flexible. Escalar para 1D, matricial para 2D (¡Cuidado!). Preferir `np.vdot` para vectores complejos.
- **np.matmul / @:** Estricto. Siempre producto matricial. Recomendado para claridad en álgebra lineal.

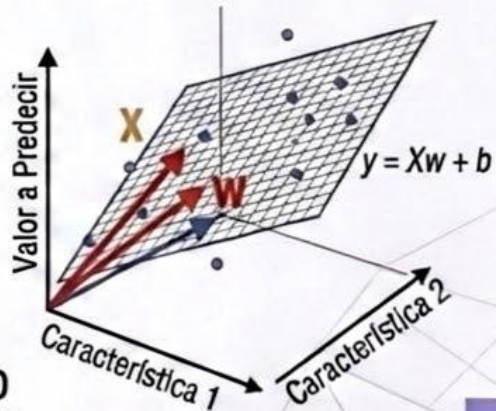
El Corazón Matemático del Modelo Predictivo

En su forma más simple, la regresión lineal (y casi cualquier modelo lineal) es una multiplicación matriz-vector:

y: La predicción final (ej. Precio de una casa)

$$\mathbf{y} = \mathbf{X} \cdot \mathbf{w} + b$$

y: La predicción final (ej. Precio de una casa) **X**: Matriz de características (Features) **w**: Vector de Pesos (w): Lo que el modelo aprende **b**: Sesgo (Bias): El intercepto



```
y_pred = X @ w + b # Forma moderna y recomendada de producto matricial  
(Es equivalente a 'y_pred = np.matmul(X, w) + b')
```

"Convertimos álgebra abstracta en predicciones reales."

Aplicación Práctica: Normalizando datos para Machine Learning

Los algoritmos matemáticos requieren que todas las variables operen en la misma escala. Usamos las operaciones vectorizadas de NumPy para lograr esto instantáneamente.



1: Min-Max (Escala 0 a 1)

$$x_{norm} = \frac{x - \min}{\max - \min}$$

```
min_val = np.min(datos)
max_val = np.max(datos)
datos_norm = (datos - min_val) /
              (max_val - min_val)
```



Z-Score (Estandarización)

$$z = \frac{x - \mu}{\sigma}$$

```
media = np.mean(datos)
desvio = np.std(datos)
datos_estandarizados = (datos -
                        media) / desvio
```

Importación y Exportación de Datos con NumPy

Importación de Datos (Leer archivos)

Archivos de Texto (CSV, TXT)

```
# Leer archivo CSV con delimitador personalizado
datos = np.loadtxt('datos.csv', delimiter=',', dtype=float)
# Usar genfromtxt para manejo de datos faltantes
datos_missing = np.genfromtxt('datos_faltantes.csv',
delimiter=',', filling_values=0)
```

Archivos Binarios NumPy (NPY, NPZ)

```
# Cargar un solo array
array_cargado = np.load('mi_array.npy')
# Cargar multiples arrays de un archivo NPZ
archivo_npz = np.load('mis_arrays.npz')
array1 = archivo_npz['arr1']
array2 = archivo_npz['arr2']
```

Nota: `genfromtxt` es más flexible que `loadtxt`.

Exportación de Datos (Guardar archivos)

Archivos de Texto (CSV, TXT)

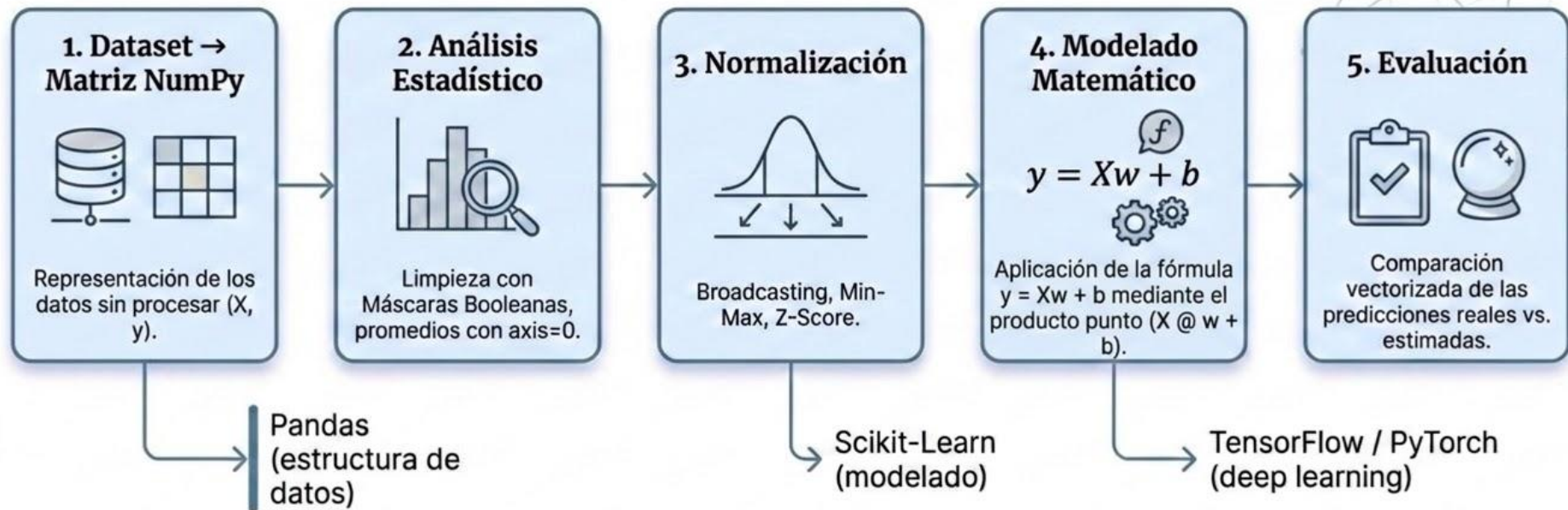
```
# Guardar array como CSV
np.savetxt('salida.csv', datos, delimiter=',')
# Guardar con formato especifico (ej: 2 decimales)
np.savetxt('salida_formato.txt', datos, fmt='%.2f')
```

Archivos Binarios NumPy (NPY, NPZ)

```
# Guardar un solo array (eficiente)
np.save('mi_array.npy', datos)
# Guardar multiples arrays en un solo archivo
np.savez('mis_arrays.npz', arr1=array1, arr2=array2)
```

Nota: Archivos binarios (`.npy`, `.npz`) son más rápidos y eficientes en espacio.

El Pipeline Completo: Del Dataset a la Predicción



La Revelación

NumPy no es solo una biblioteca para escribir menos código. Es la infraestructura matemática que traduce la realidad del mundo (datos) al idioma nativo que comprenden los algoritmos (álgebra lineal). $\text{np.dot}(X, w) + b$